

PHASE-1 RTL Design

AXI4-Lite is a simplified, non-bursting subset of the AMBA AXI protocol designed specifically for simple control-and-status-register communication. It eliminates the complex burst tracking, out-of-order execution, and data interleaving of full AXI, making it ideal for the memory-mapped slave you are verifying.

The protocol separates reads and writes into five unidirectional, independent channels:

- **Write Address (AW):** Master sends the target memory address and control signals.
- **Write Data (W):** Master sends the actual data and byte strobes (**WSTRB**).

The **Write Strobe (WSTRB)** signal acts as a byte-enable mask. It tells the slave exactly which bytes within the 32-bit (or 64-bit) data bus actually contain valid data that should be written to the register or memory.

Because AXI4-Lite only supports full data-bus-width transfers, the master uses **WSTRB** to perform smaller, partial-word writes (like updating a single 8-bit byte inside a 32-bit register) without overwriting the adjacent bytes.

How It Works

There is exactly one **WSTRB** bit for every 8 bits (one byte) of **WDATA**. For a standard 32-bit AXI4-Lite bus, **WDATA** is 32 bits wide, so **WSTRB** is 4 bits wide.

- **WSTRB[0]** controls **WDATA[7:0]** (Byte 0 - Lowest byte)
- **WSTRB[1]** controls **WDATA[15:8]** (Byte 1)
- **WSTRB[2]** controls **WDATA[23:16]** (Byte 2)
- **WSTRB[3]** controls **WDATA[31:24]** (Byte 3 - Highest byte)

If **WSTRB[n]** is **1**, the slave must write that specific byte into the register. If **WSTRB[n]** is **0**, the slave must ignore that byte and leave the existing register value intact.

Common Examples (32-bit Bus)

- **WSTRB = 4'b1111**: A full 32-bit word write. All 4 bytes are updated.
- **WSTRB = 4'b0001**: A single byte write. Only the lowest 8 bits are updated. The upper 24 bits of the register remain unchanged.
- **WSTRB = 4'b0011**: A half-word (16-bit) write. The lower 16 bits are updated.
- **WSTRB = 4'b0000**: A null write. The master is sending data, but nothing should be written to the register.

Why This is Critical for Your UVM Verification

If your slave ignores the **WSTRB** signal and blindly writes all 32 bits of **WDATA** whenever a write occurs, it is a critical protocol violation.

In your UVM environment, you must verify this behavior:

- **Sequences:** Your constrained-random sequences should generate writes with randomized **WSTRB** values (e.g., partial writes, sparse strobes like **4'b1010**, and all-zero strobes).
- **RTL Implementation:** Your SystemVerilog slave logic needs a byte-by-byte write enable condition. (e.g., `if (WSTRB[0]) reg_data[7:0] <= WDATA[7:0];`).
- **Scoreboard:** Your UVM scoreboard must use the **WSTRB** value from the monitor to accurately predict the expected state of the register. It must apply a bitwise mask to combine the old register value with the new **WDATA** based on the strobe bits.

- **Write Response (B):** Slave sends a 2-bit status response indicating write success or failure.
- **Read Address (AR):** Master sends the target memory address and control signals.
- **Read Data (R):** Slave sends the requested data and a 2-bit status response.

One correction to keep in mind from the formal AXI specification: each channel has its own independent **VALID/READY** handshake, and a transfer occurs on a clock edge when both are high. AXI4-Lite also has no bursts; each access is a single transfer.

The important rule is:

Once **RVALID** is asserted, the slave cannot simply withdraw the response because **RREADY** is low.

This is exactly the sort of behavior our assertions and backpressure tests will check. The same VALID-until-READY principle applies to the other channels.

It has five independent channels:

Write

1. Write Address Channel

AWADDR
AWVALID
AWREADY

2. Write Data Channel

WDATA
WSTRB
WVALID
WREADY

3. Write Response Channel

BRESP
BVALID
BREADY

Read

4. Read Address Channel

ARADDR
ARVALID
ARREADY

5. Read Data Channel

RDATA
RRESP
RVALID
RREADY

The really important case: AW first, W later

AXI allows the address and data channels to operate independently.

So this is legal:

Cycle 1:

AWVALID = 1

AWREADY = 1

→ address accepted

Cycle 2:

WVALID = 1

WREADY = 1

→ data accepted

Internally, our slave needs to remember:

aw_received = 1

and store:

awaddr_reg = AWADDR

Then when W arrives:

w_received = 1

Now:

aw_received && w_received

becomes true.

Only then should the register actually be updated.

This is one reason our RTL needs some internal state.

8. The opposite: W first, AW later

This is equally important.

Cycle 1:

WVALID = 1

WREADY = 1

→ data accepted

Cycle 2:

AWVALID = 1

AWREADY = 1

→ address accepted

Now the slave has:

stored_data = WDATA

stored_strobes = WSTRB

stored_address = AWADDR

Once both are available:

address_received && data_received

we perform the write.

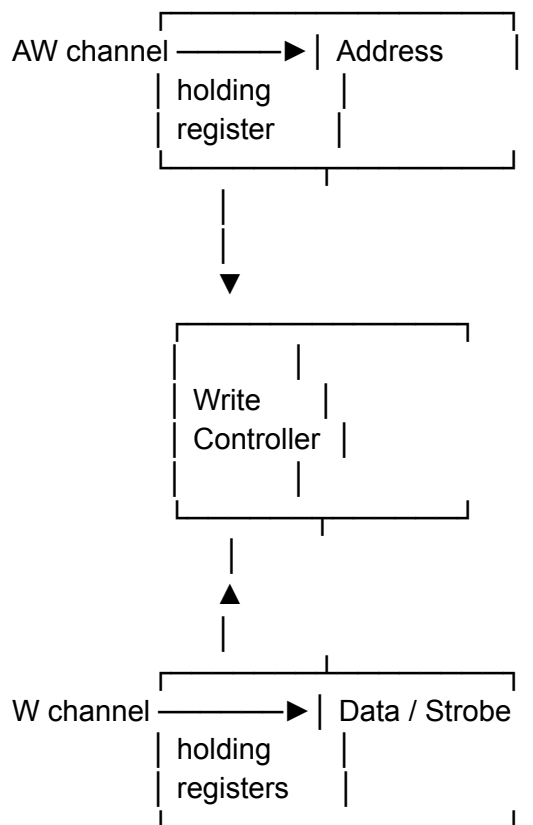
This is a **very important verification scenario**.

A naive AXI implementation can accidentally assume:

AW arrives first

and fail when W arrives first.

Our constrained-random UVM environment should specifically find this kind of bug.



VALID/READY Handshaking (Critical for Assertions & Backpressure): Every channel uses a strict two-way handshake. The source drives the payload and asserts **VALID**. The destination asserts **READY** when it can accept the payload. The transfer occurs on the rising clock edge when both are HIGH.

- **Backpressure:** Your slave tests backpressure by driving **READY** LOW, forcing the master to wait.
- **Assertion Rules:**
 - A source **must not** wait for **READY** before asserting **VALID**.
 - Once **VALID** is asserted, it **must remain HIGH** (along with the data/address payload) until the handshake completes. The source cannot change its mind and drop the transaction.
 - A destination *is permitted* to wait for **VALID** before asserting **READY**.

Addressing and Data Rules (Critical for Sequence Constraints)

- **Single Transfers Only:** Every transaction is a single transfer. Burst length is always 1.
- **Bus Width:** The data bus (**WDATA**, **RDATA**) must be exactly 32-bit or 64-bit.
- **Alignment:** All accesses must be aligned to the width of the data bus. An unaligned address is a protocol violation.
- **Byte Strokes (**WSTRB**):** Masters use strobes to indicate which specific bytes within the data word are valid. Your slave must respect these strobes to avoid corrupting adjacent bytes in a register.

Responses (Critical for Error Handling and Invalid Addresses) Your slave must return a 2-bit response for every transaction via **BRESP** (for writes) or **RRESP** (for reads).

- **OKAY (2'b00)**: Normal access success.
- **EXOKAY (2'b01)**: Exclusive access success (Not supported in AXI4-Lite; usually falls back to OKAY).
- **SLVERR (2'b10)**: Slave Error. Your slave returns this when the master attempts to access an **invalid or unmapped address**, or attempts an unauthorized write (e.g., writing to a read-only register).
- **DECERR (2'b11)**: Decode Error. Typically generated by the system interconnect when no slave exists at the requested address.

Reset Behavior (Critical for Reset Sequences) The protocol relies on an active-low synchronous reset (**ARESETn**).

- During reset, all **VALID** signals across all channels **must** be driven strictly LOW.
- **READY** signals can be driven to any value during reset. It is a common design practice to assert slave **READY** signals HIGH during reset so the slave can accept a transaction on the very first cycle after reset deasserts.

In digital design, **backpressure** is a flow-control mechanism where a receiving component (the slave) signals a sending component (the master) to temporarily stop sending data because it cannot process it fast enough.

It is the hardware equivalent of saying, *"Hold on, my hands are full—wait a second before handing me the next item."*

Here is how it works and why it matters in your AXI4-Lite project:

The Problem It Solves

Imagine a high-speed processor (Master) trying to write configuration data to a slower peripheral device (Slave). If the Master just blasts data continuously, the Slave's internal buffers will overflow, and data will be lost.

How Backpressure Works in AXI4-Lite

In AXI, backpressure is implemented using the **READY** signal.

1. The Master puts data on the bus and asserts **VALID = 1** (saying: "I have valid data for you").
2. The Slave might be busy doing something else. It asserts **READY = 0** (saying: "I am not ready to accept it yet").
3. This **READY = 0** is the **backpressure**. It "pushes back" against the flow of data.
4. Because of the AXI protocol rules, the Master is now trapped: it must hold the data steady and keep **VALID = 1** indefinitely until the Slave finally asserts **READY = 1**.

AXI4-Lite is a simplified AXI interface intended primarily for **control/status register access**.

Why are we deliberately testing backpressure?

This is one of the important parts of the project.

A beginner test might do:

VALID = 1

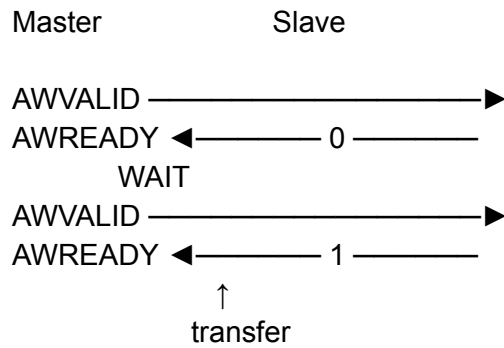
READY = 1

→ transaction

and everything works.

But real hardware cannot assume the other side is always ready.

For example:

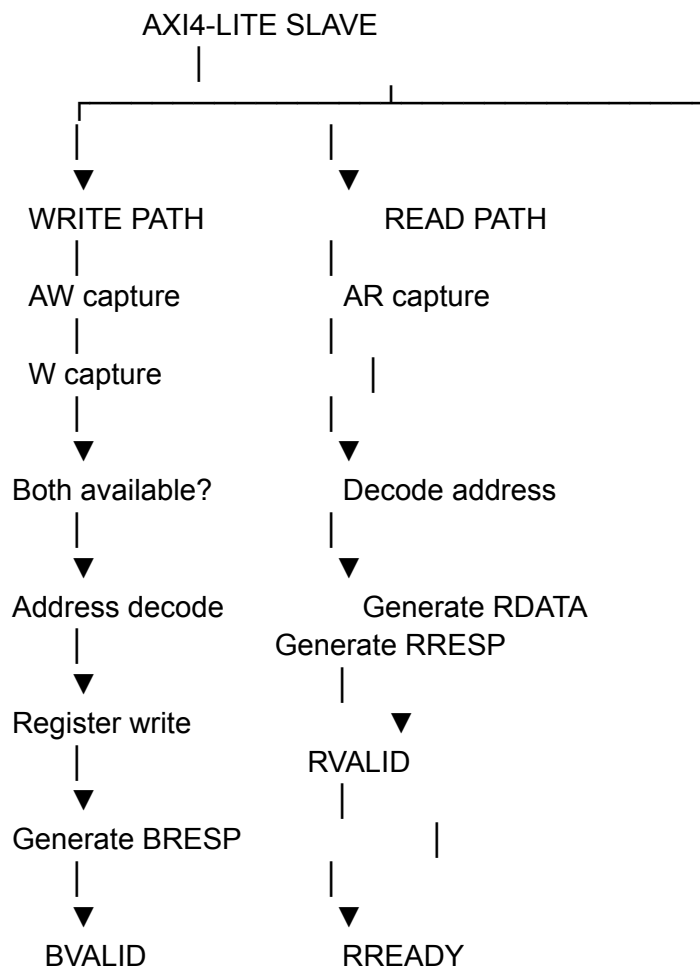


Our verification environment should intentionally manipulate READY signals so that the DUT has to wait.

That allows us to verify that:

- data isn't lost
- addresses aren't lost
- VALID remains asserted correctly
- responses aren't lost
- transactions don't complete prematurely
- the slave recovers correctly once READY becomes asserted

This is much more meaningful than simply testing a few reads and writes.



The UVM environment will generate transactions such as:

```
axi_transaction
{
    operation = WRITE;
    address   = 0x08;
    data      = 0x12345678;
}
```

The **sequence** generates these transactions.

The **sequencer** delivers them to the driver.

The **driver** converts the transaction into actual AXI signals.

For example:

axi_transaction



Driver



AWADDR = 0x08

WDATA = 0x12345678

AWVALID = 1

WVALID = 1

The DUT processes the signals.

Then the **monitor** observes what actually happened.

The **scoreboard** compares:

Expected behavior

VS

Observed behavior

Instead of manually writing:

```
write(0x00, 1);
```

```
write(0x04, 2);
```

```
read(0x00);
```

```
read(0x04);
```

we'll create randomized transactions.

Something conceptually like:

WRITE 0x00 → 0xA13F...

READ 0x08

WRITE 0x0C → 0x91...

READ 0x00

WRITE invalid address

READ invalid address

...

And constrain the randomization so we generate meaningful combinations.

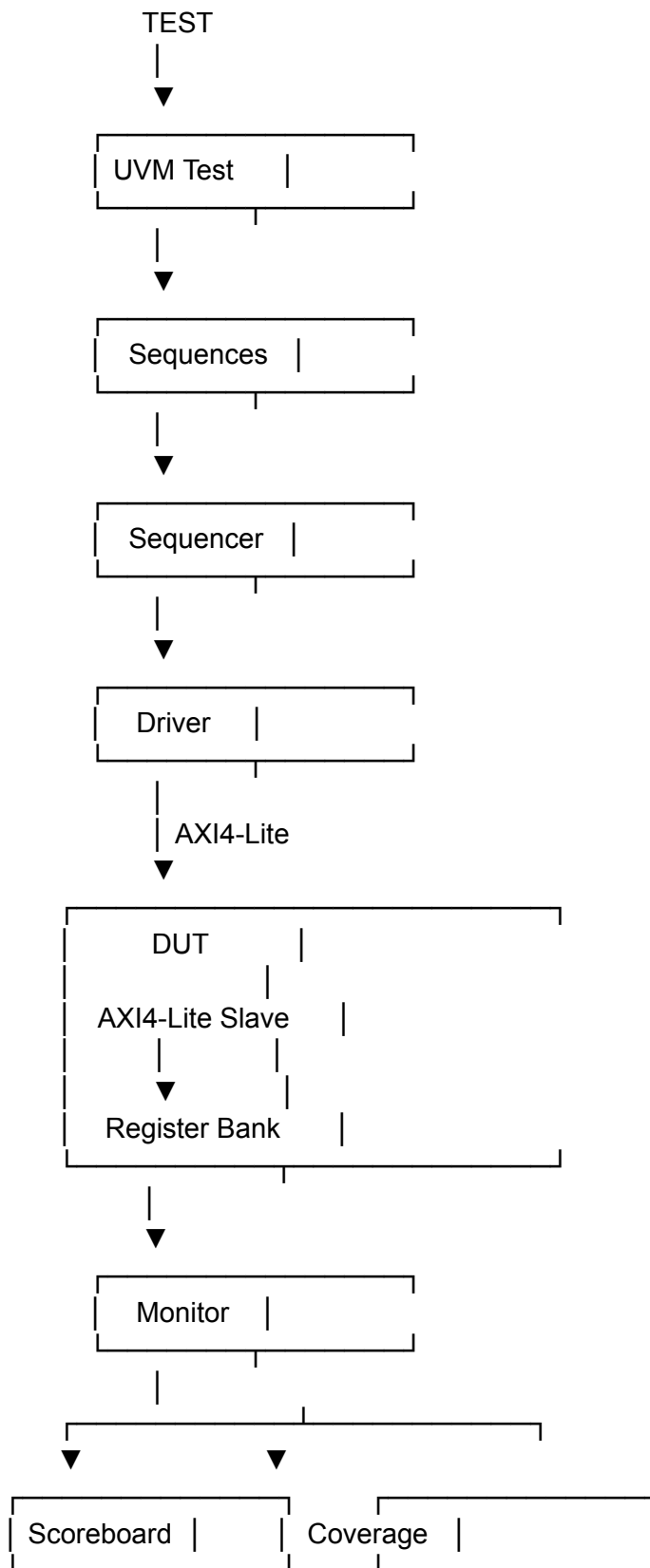
For example:

70% valid addresses

20% invalid addresses

10% boundary addresses

The exact distributions can be decided later.



- + SystemVerilog Assertions
- + Randomized Regression
- + Functional Coverage
- + Code Coverage

We'll build it progressively:

Phase 1 — Understand AXI4-Lite

We'll establish exactly how:

- AW channel works
- W channel works
- B channel works
- AR channel works
- R channel works
- VALID/READY works
- independent channels work
- backpressure works

Phase 2 — Define our peripheral

We'll decide:

Data width = 32-bit

Address width = 32-bit

Register count = 4–8

and define the register map.

Phase 3 — Build the RTL

First get:

RESET

↓

READ

↓

WRITE

↓

VALID/READY

↓

BACKPRESSURE

↓

ERROR RESPONSES

working.

Phase 4 — Basic simulation

Before UVM, we'll directly verify the RTL with a simple SystemVerilog testbench.

This is important because if something fails later, we'll know whether the problem is:

DUT

or

UVM environment

Phase 5 — Build UVM

Then:

transaction

interface

sequence

sequencer

driver

monitor
agent
scoreboard
environment
test

Phase 6 — Add constrained-random testing

Generate randomized:

- addresses
- read/write operations
- data
- stalls
- transaction ordering

Phase 7 — Add assertions

Protocol properties such as:

VALID must remain asserted until READY
and other AXI correctness rules.

Phase 8 — Add coverage

Functional coverage + cross coverage.

Phase 9 — Regression

Run many randomized tests/seeds and collect:

Tests

↓

Pass/Fail

↓

Coverage

↓

Coverage gaps

↓

New constraints/tests

↓

Regression

Phase 10 — Coverage convergence

The final goal isn't simply:

"The tests passed."

It is:

Functional correctness + protocol correctness + meaningful coverage + no unexplained coverage gaps.

When reset is asserted:

RESET

↓

CONTROL = 0

STATUS = 1

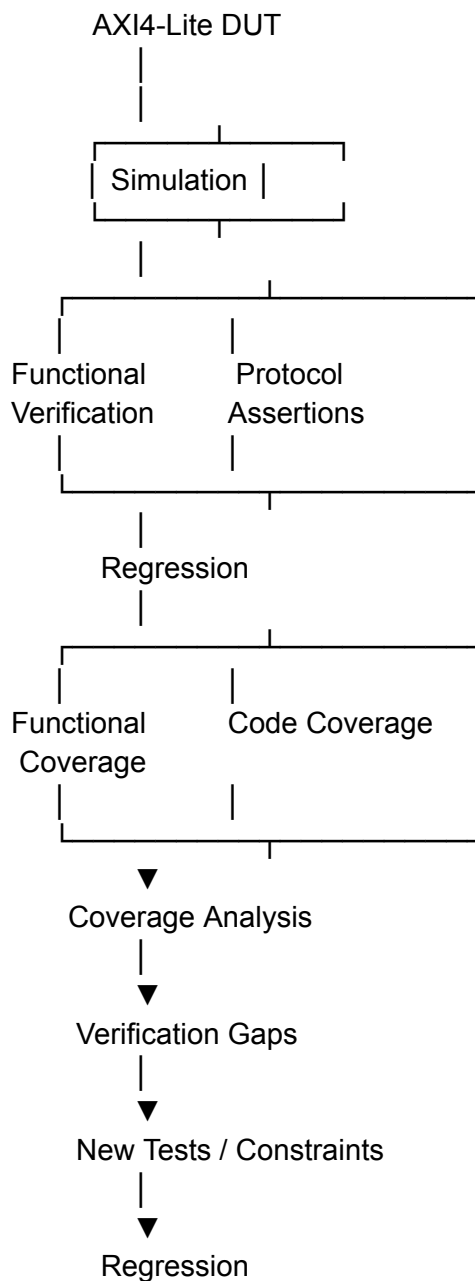
DATA_IN = 0
DATA_OUT = 0
CONFIG = 0
COUNT = 0
VERSION = 0x00010000
SCRATCH = 0

And the AXI interface returns to an idle state.

We will later test reset:

1. Before any transaction
2. Between transactions
3. During a pending read
4. During a pending write

The exact behavior for reset during an active transaction will be specified carefully when we write the protocol assertions.



FSMs associated with it

One important clarification first: AXI4-Lite does not prescribe a particular FSM implementation. We are designing our own internal FSMs to manage the five independent AXI channels correctly.

For our project, I recommend separate read and write control FSMs, rather than one giant FSM.

There is actually another perfectly valid way to implement this.

Instead of:

WRITE_IDLE

WAIT_W

WAIT_AW

WRITE_EXEC

B_RESPONSE

we could use flags:

aw_received

w_received

and a smaller FSM:

WRITE_IDLE

WRITE_EXEC

B_RESPONSE

For example:

```
if (AWVALID && AWREADY)
```

```
    aw_received <= 1;
```

```
if (WVALID && WREADY)
```

```
    w_received <= 1;
```

```
if (aw_received && w_received)
```

```
    execute_write();
```

This is often cleaner for AXI-Lite because the channels are independent.

For our project, I prefer the flag-based approach internally, while thinking about it as the four conditions:

00 → nothing

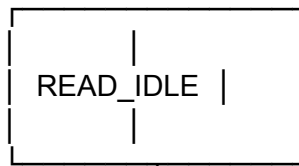
10 → AW only

01 → W only

11 → both

It will make the RTL less unnecessarily state-heavy and give us a good opportunity to discuss

FSM vs control-flag architecture in an interview.



ARVALID
& ARREADY

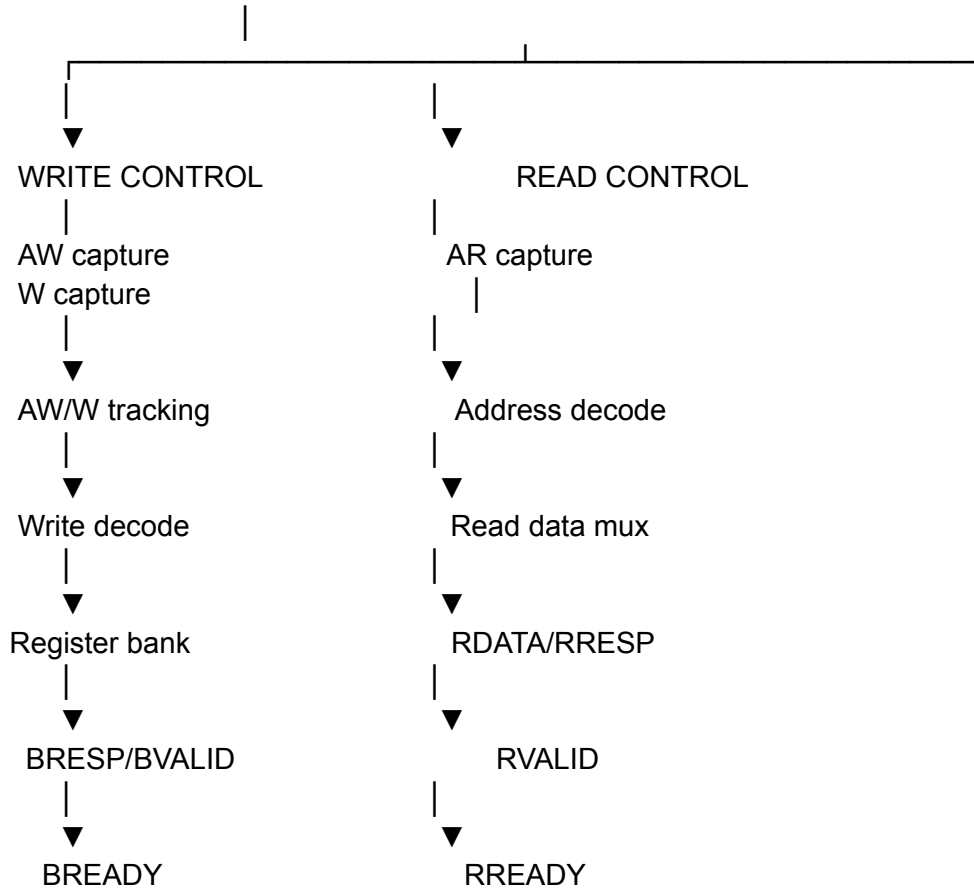


RVALID
& RREADY

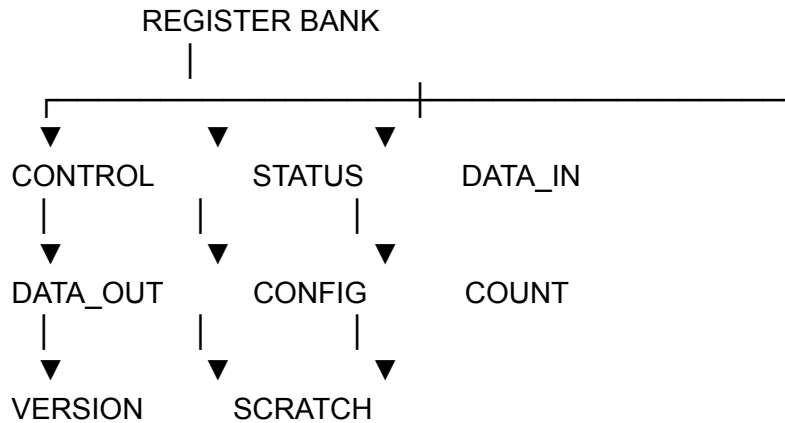


READ_IDLE

AXI4-LITE SLAVE



And underneath:



One subtle issue: when should **READY** be asserted?

This is something we should decide **before coding**.

A simple implementation could say:

AWREADY = 1 whenever we can accept an address

WREADY = 1 whenever we can accept data

ARREADY = 1 whenever read path is idle

But there are tradeoffs.

For example, suppose we've already captured:

AWADDR

but haven't received W yet.

Should:

AWREADY

remain high?

No.

Otherwise the master could send another address and we'd have nowhere to store it.

Similarly:

AW captured

→ AWREADY = 0

W captured

→ WREADY = 0

until the current transaction has completed.

This means our slave is effectively designed to support **one outstanding write transaction** and **one outstanding read transaction** at a time.

That's perfectly reasonable for this project and keeps the RTL manageable.

21. The architecture we're choosing

So let's lock this in:

Write path

One outstanding write

AW → holding register

W → holding register

AW_received

W_received

when both:

execute write

generate B response

hold BVALID until BREADY

Read path

One outstanding read

AR → holding register

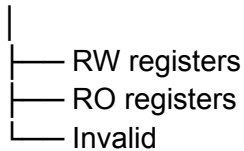
decode

generate RDATA/RRESP

hold RVALID until RREADY

Register bank

Address decoder



IMP EXAMPLE

Transaction we're trying to perform

Let's say the master wants:

WRITE 0x08 = 0xDEADBEEF

Recall:

0x08 = DATA_IN

So the final expected result is:

DATA_IN = 0xDEADBEEF

BRESP = OKAY

The master sends:

AWADDR = 0x08

WDATA = 0xDEADBEEF

WSTRB = 4'b1111

But deliberately, **W arrives before AW**.

2. Internal registers

Our slave needs to remember what it has received.

We'll have:

awaddr_reg

wdata_reg

wstrb_reg

aw_received

w_received

Think of these as our slave's temporary storage.

Initially:

awaddr_reg = X
wdata_reg = X
wstrb_reg = X

aw_received = 0
w_received = 0

And we're in:

WRITE_IDLE

3. Cycle 0 — Initial state

Before anything happens:

WRITE_IDLE

aw_received = 0
w_received = 0

AWREADY = 1

WREADY = 1

BVALID = 0

The slave is basically saying:

"I have nothing pending. Give me an address or data."

4. Cycle 1 — W arrives first

The master presents:

WVALID = 1

WDATA = DEADBEEF

WSTRB = 1111

But:

AWVALID = 0

The slave has:

WREADY = 1

Therefore:

WVALID && WREADY

↓
1

So the W transaction occurs on the rising edge.

Before edge

WVALID = 1

WREADY = 1

WDATA = DEADBEEF

At edge

W transfer

After edge

Our internal state becomes:

wdata_reg = DEADBEEF

wstrb_reg = 1111

w_received = 1

aw_received = 0

So:

WRITE SIDE

Address: NOT RECEIVED

Data: RECEIVED ✓

wdata_reg = DEADBEEF

The controller is now effectively in:

WAIT_FOR_AW

5. What happens to WREADY now?

This is important.

We've already captured the write data.

We don't have another storage slot.

Therefore:

WREADY = 0

We don't want the master sending another W transaction.

So:

Before W

WREADY = 1

After W

WREADY = 0

The slave is saying:

"I already have the data for the current transaction. I can't accept another one yet."

6. Cycle 2 — AW arrives

Now the master presents:

AWVALID = 1

AWADDR = 0x08

Our slave has:

AWREADY = 1

Therefore:

AWVALID && AWREADY

↓
1

The address transfers.

At the rising edge:

awaddr_reg <= 0x08

aw_received <= 1

Now our internal state is:

aw_received = 1

w_received = 1

awaddr_reg = 0x08

wdata_reg = DEADBEEF

wstrb_reg = 1111

We now have everything required.

7. The important condition

Our control logic effectively sees:

aw_received && w_received

which is:

1 && 1

↓

1

Therefore:

The write transaction is complete from the input-channel perspective.

Now we can execute it.

8. Cycle 3 — Write execution

We have:

Address = 0x08

Data = DEADBEEF

WSTRB = 1111

Decode:

0x08

↓

DATA_IN

Access type:

DATA_IN = RW

Therefore:

DATA_IN <= DEADBEEF

Because:

WSTRB = 1111

all four bytes are enabled.

So:

DATA_IN

↓

0xDEADBEEF

At the same time, we determine:

BRESP = OKAY

9. Now we enter the response state

The slave asserts:

BVALID = 1

BRESP = OKAY

The key thing is:

The register write has already happened.

The B channel is simply telling the master:

"Your write has completed successfully."

So our conceptual state becomes:

B_RESPONSE

10. But now the master applies backpressure

Here's the interesting part.

The master says:

BREADY = 0

So:

BVALID = 1

BREADY = 0

Therefore:

BVALID && BREADY = 0

No response transfer occurs.

11. Cycle 4

Our slave must **keep**:

BVALID = 1

BRESP = OKAY

So:

Cycle 4

BVALID _____ 1

BREADY _____ 0

BRESP _____ OKAY

The FSM remains:

B_RESPONSE

12. Cycle 5

The master still isn't ready:

BREADY = 0

Again:

BVALID = 1

Nothing happens.

The slave continues waiting.

Cycle 5

BVALID = 1

BREADY = 0

STATE = B_RESPONSE

13. Cycle 6 — Master finally becomes ready

Now:

BREADY = 1

while:

BVALID = 1

Therefore:

BVALID && BREADY

↓

1

The response transfers.

TRANSFER
↓

BVALID = 1 —————●
 BREADY = 1 —————●
 BRESP = OKAY

The write transaction is now finished.

14. Cycle 7 — Return to idle

Now we can clear our internal tracking:

aw_received <= 0

w_received <= 0

and return to:

WRITE_IDLE

Then:

AWREADY = 1

WREADY = 1

The slave can accept a new write.

15. Entire transaction in one table

Here's the complete sequence:

Cycle	Event	aw_received	w_received	State
0	Idle	0	0	WRITE_IDLE
1	W handshake	0	1	WAIT_AW
2	AW handshake	1	1	WRITE_EXEC
3	Register updated	1	1	B_RESPONSE
4	B stalled	1	1	B_RESPONSE
5	B stalled	1	1	B_RESPONSE
6	B handshake	1	1	B_RESPONSE
7	Cleanup	0	0	WRITE_IDLE

And the important values:

Cycle 1:

WDATA = DEADBEEF

↓

wdata_reg = DEADBEEF

Cycle 2:

AWADDR = 08

↓

awaddr_reg = 08

Cycle 3:

08 → DATA_IN



DATA_IN = DEADBEEF



BRESP = OKAY

BVALID = 1

Cycle 4-5:

BREADY = 0



BVALID remains 1

Cycle 6:

BREADY = 1



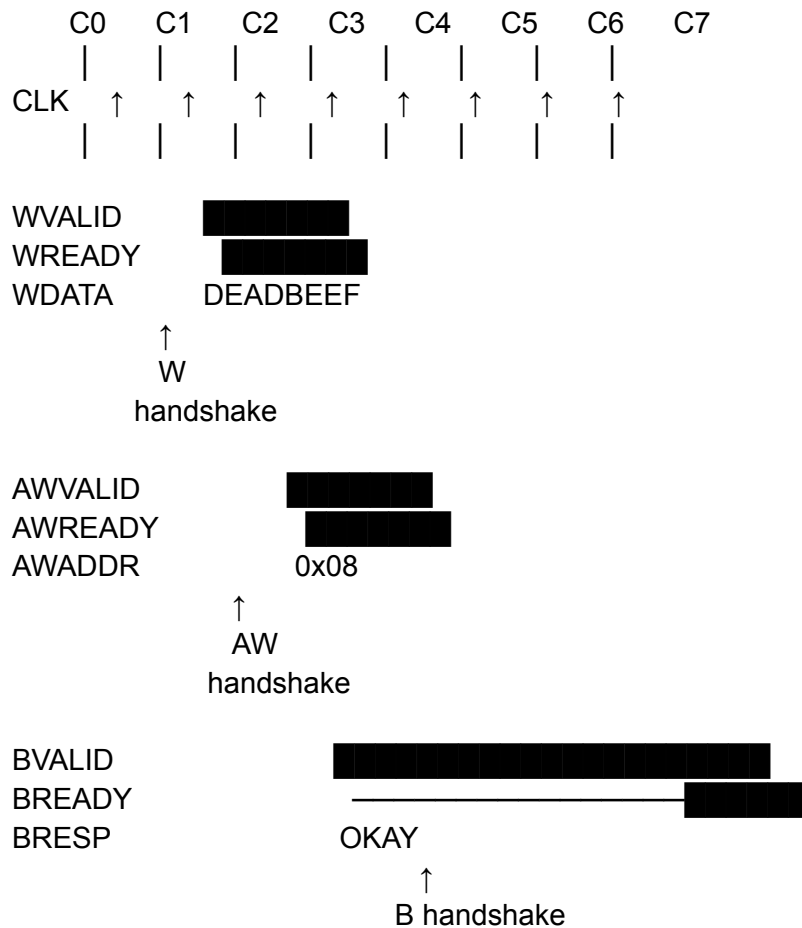
B handshake

Cycle 7:

Return to IDLE

16. Now let's look at the actual signal timeline

This is the most useful picture:





This one diagram captures almost the entire write-control problem.

17. Now let's ask: what if W never arrives?

Suppose:

AW handshake

happens, but then the master takes 10 cycles before presenting W.

Our slave does:

aw_received = 1

w_received = 0

and waits.

It **must not**:

perform the write

generate BVALID

lose the address

accept another AW

So:

AWREADY = 0

WREADY = 1

BVALID = 0

until W arrives.

That's another important verification scenario.

18. What if AW never arrives?

Same logic, reversed.

After W:

aw_received = 0

w_received = 1

Then:

AWREADY = 1

WREADY = 0

BVALID = 0

We wait.

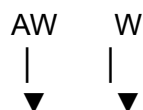
This is why the **two holding registers + two flags** architecture works nicely.

19. Now the really important insight

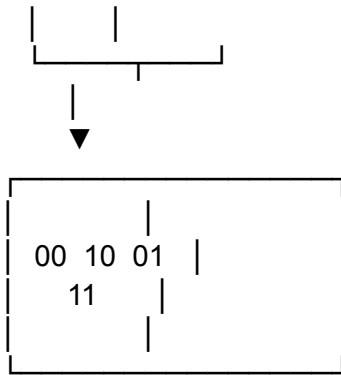
The FSM isn't really "AW state → W state."

Because AXI allows both orders.

The real state of the write controller is determined by:



received? received?



You can think of it as a **2-bit state machine**:

aw_received w_received

00 → nothing received

10 → address received

01 → data received

11 → both received → execute

This is actually a very clean way to understand AXI's independent channels.

20. Why this is useful for UVM

Now imagine we're writing our randomized sequence.

We can explicitly generate:

Scenario 1:

AW + W same cycle

Scenario 2:

AW → wait 1 cycle → W

Scenario 3:

AW → wait 5 cycles → W

Scenario 4:

W → wait 1 cycle → AW

Scenario 5:

W → wait 5 cycles → AW

Then add:

BREADY delay = 0 cycles

BREADY delay = 1 cycle

BREADY delay = 5 cycles

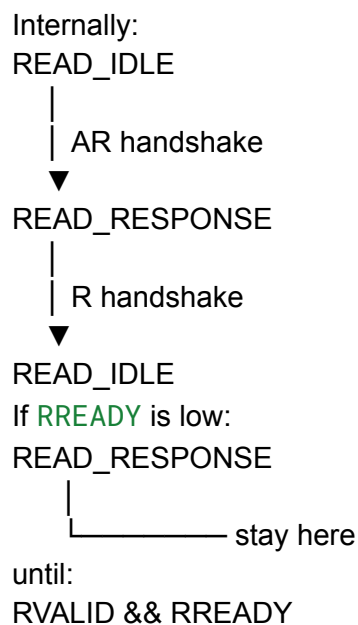
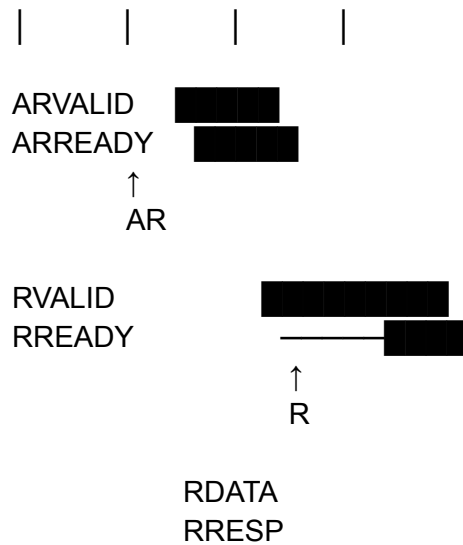
We're effectively testing every path through our control logic.

That is where the **constrained-random + coverage** portion of your project becomes meaningful.

21. The read side is much simpler

For comparison:

C0 C1 C2 C3



22. One thing I want you to notice

There are **three different concepts** here:

1. Channel handshake

VALID && READY

2. Transaction completion

For a write:

AW received

+

W received

+

B response accepted

For a read:

AR received

+

R response accepted

3. Register operation

For a write:

AW + W received



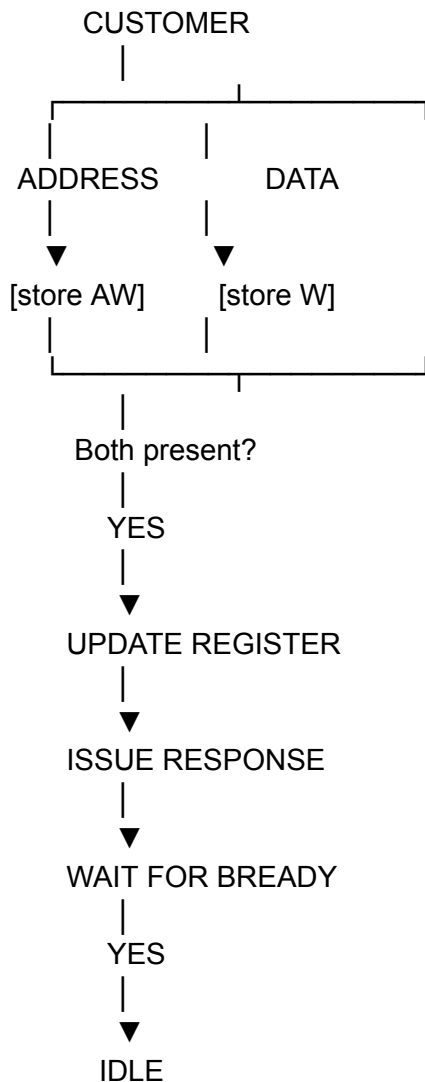
register update

These are **not the same event**.

That's an extremely important distinction in AXI verification.

23. Our final write-control mental model

Think of the slave as a small clerk handling one customer:



That is essentially our entire write datapath.